

Docu-Intel

Arquitectura actual e ideal para una plataforma OCR segura con IA intermedia local. Fecha: 14/05/2026.

Objetivo

- Convertir carpetas con presupuestos, pedidos, facturas, planos, imagenes, PDFs y Excels en una base documental consultable.
- Guardar texto OCR, bloques, entidades, datos estructurados, fuentes, confianza y relaciones documentales.
- Servir a una herramienta externa con IA mediante una API segura, sin SQL libre y con permisos aplicados antes de entregar datos.
- Evitar fugas: un tecnico solo ve su hotel/cadena y no recibe precios si su perfil no lo permite.

Arquitectura actual



Componentes principales

Frontend React	Interfaz interna: dashboard, documentos, visor, busqueda, jobs, presupuestos, pedidos, planos y administracion.
Backend FastAPI	API central: autenticacion, permisos, ingesta, busqueda, integracion, auditoria y redaccion de datos sensibles.
PostgreSQL + pgvector	Datos estructurados, texto OCR, bloques, entidades, documentos, jobs, permisos, auditoria y embeddings.
Redis	Broker de Celery, cache y soporte operativo.
Celery workers	Procesamiento asincrono de documentos y tareas pesadas.
Watcher	Vigila carpetas 24h, detecta archivos estables y registra eventos de ingesta.
/data/files	Almacenamiento local de originales. No se borran fisicamente por defecto.
/integrations/v1	API para la otra herramienta/IA. Expone tools controladas y contexto saneado.

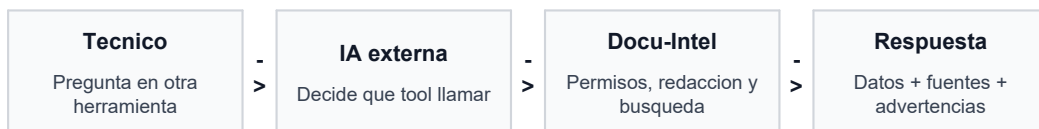
Flujo de procesamiento documental

- El archivo entra por subida manual, API de integracion o carpeta vigilada.
- Se espera a que el archivo deje de cambiar para evitar procesar copias incompletas.
- Se calcula SHA256; si ya existe, se registra como duplicado y apunta al documento canonico.
- El original se guarda en /data/files usando hardlink si es posible o copia normal como fallback.
- Se crea el documento, el job y el evento operativo de ingesta.
- Celery enruta el trabajo a text_fast, ocr_heavy, embeddings o maintenance.
- El parser extrae texto, bloques, paginas, entidades, datos estructurados y chunks.
- Los embeddings se generan con servidor local compatible OpenAI o fallback local si esta activado.
- El documento queda processed, needs_review, failed o duplicate con trazabilidad en BD.

Colas recomendadas

text_fast	PDF digital, Excel, CSV, TXT	Trabajos rapidos que no deben esperar al OCR pesado.
ocr_heavy	Imagenes, PDFs escaneados, planos	Consumen CPU/RAM y deben poder escalarse aparte.
embeddings	Vectorizacion de chunks	Permite recalcular embeddings sin bloquear OCR.
maintenance	Escaneos, tareas periodicas	Separacion de tareas operativas y procesamiento documental.

API segura para IA intermedia



- La IA externa no accede a PostgreSQL ni genera SQL libre.
- Cada llamada usa X-DocuIntel-API-Key y X-Technician-Id.
- Docu-Intel resuelve la política del tecnico antes de consultar datos.
- Si no puede ver precios, se eliminan campos monetarios y el OCR se redacta como [IMPORTE OCULTO].
- Si el documento pertenece a otro hotel/cadena, se responde como no encontrado dentro del scope autorizado.
- Todas las respuestas deben incluir fuentes cuando vienen de documentos.

Aislamiento por hotel/cadena

La regla critica es que ningun cliente, tecnico ni IA reciba documentos fuera de su scope autorizado. Los tags sensibles siempre prevalecen sobre permisos de hotel o cadena.

hotel_chains	Cadena hotelera.
hotels	Hotel asociado a una cadena.
document_access_metadata	Asigna documento a cadena/hotel, estado de asignacion y tags.
folder_assignment_rules	Reglas por ruta para asignar automaticamente documentos.
access_groups	Define hoteles, cadenas, precios permitidos y tags bloqueados.
access_group_members	Relaciona usuarios internos o tecnicos externos con grupos.

Ejemplos de politica

- Cliente Hotel A: solo ve documentos asignados a Hotel A.
- Cliente de cadena: puede ver varios hoteles de su cadena, nunca otra cadena.
- Gestor interno: puede tener `allow_all_hotels=true`, pero `denied_tags=['contabilidad']` bloquea facturas sensibles.
- Admin: acceso total para operacion, revision y recuperacion.

Arquitectura ideal para produccion

Servidor	Linux dedicado. Docker Compose es suficiente al inicio; Kubernetes o varios servidores si el volumen crece mucho.
PostgreSQL	SSD/NVMe, backups, indices, autovacuum ajustado y monitorizacion. Separarlo si la carga OCR crece.
Storage	/data/input para entrada vigilada y /data/files para originales inmutables. Backups de BD y originales.
OCR	Workers OCR separados. Para mucho volumen, varios workers o GPU. Limitar concurrencia para proteger RAM.
IA local	Servidor separado con Ollama, LM Studio, vLLM o llama.cpp. Docu-Intel solo envia contexto filtrado.
Seguridad	JWT interno, API key de servicio, X-Technician-Id, politicas por hotel/cadena/tags y redaccion backend.
Operacion	Panel de colas, eventos, disco, errores, reprocesado parcial, auditoria y alertas.

Resumen ejecutivo

Docu-Intel debe funcionar como una central documental OCR segura: ingesta documentos, extrae texto y datos, aplica permisos, redacta informacion sensible y expone tools controladas para que otra IA responda con fuentes sin inventar datos. La parte mas critica no es solo el OCR, sino no entregar nunca a la IA datos que el usuario final no pueda ver.